

Contents

□ Greedy Algorithms — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	6
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	7
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	8
SECTION 11 — Pattern Variations	8
SECTION 12 — Comparison With Other Patterns	9
SECTION 13 — Problem Classification	9
SECTION 14 — 30 Interview Problems	10
SECTION 15 — Common Mistakes	12
SECTION 16 — Optimization Techniques	13
SECTION 17 — Multiple Language Templates	13
SECTION 18 — Dry Runs	15
SECTION 19 — Advanced Concepts	16
SECTION 20 — Senior Engineer Insights	16
SECTION 21 — Cheat Sheet	16
SECTION 22 — Revision Notes (5-Minute Review)	17
SECTION 23 — Practice Roadmap	17
SECTION 24 — Memory Tricks	17
SECTION 25 — Final Summary	17

□ Greedy Algorithms — Complete Interview Handbook

Pattern #27 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Greedy Algorithms section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

A Greedy algorithm builds a solution by making the **locally optimal choice at every step**, never reconsidering or backtracking on past decisions, and relies on a **proof** (usually an exchange argument or matroid-theoretic argument) that this sequence of local choices provably leads to a globally optimal solution — without ever needing to explore alternative paths, unlike Backtracking or DP.

1.2 Why Was This Pattern Invented?

Many optimization problems have a special structure where committing early to the “obviously best” immediate choice never forecloses a better overall outcome — for these problems, exhaustively exploring all alternatives (Backtracking) or tracking overlapping subproblems (DP) is wasted effort; a single, provably-correct pass suffices. Greedy formalizes recognizing and exploiting this special structure for maximum efficiency (typically $O(n \log n)$, dominated by an initial sort).

1.3 Real Intuition Behind The Pattern

Imagine **making change with the fewest coins using standard currency denominations (quarters, dimes, nickels, pennies)** — always taking the largest denomination that doesn’t overshoot the remaining amount works perfectly for this specific coin system. But this greedy strategy would **fail** for an arbitrary/adversarial coin system (e.g., denominations $\{1, 3, 4\}$ making change for 6 — greedy picks $4+1+1=3$ coins, but $3+3=2$ coins is optimal) — this contrast is the entire point: greedy only works when the problem has a provable structural guarantee, not just because “it seems reasonable.”

1.4 Mental Model

“Can I prove that making the locally best choice right now never prevents reaching the globally best outcome later?” This proof (usually via an **exchange argument**: “if an optimal solution didn’t make this greedy choice, I can modify it to make this choice without making it worse”) is the entire justification for the pattern — without it, a greedy approach is just an unproven heuristic that might be wrong.

1.5 Visual Explanation

Activity Selection (maximize number of non-overlapping intervals):
Intervals sorted by END time: $[1,3]$, $[2,4]$, $[3,5]$, $[6,7]$

Greedy: always pick the next interval with the earliest end time that doesn't overlap the last picked one
Pick $[1,3]$ (earliest end). Next candidate $[2,4]$ overlaps (starts before 3 ends) → skip.
Next candidate $[3,5]$: starts at 3, previous ends at 3 → no overlap (assuming touching is OK) → pick $[3,5]$
Next candidate $[6,7]$: starts at 6, previous ends at 5 → no overlap → pick $[6,7]$.

Result: $\{[1,3], [3,5], [6,7]\}$ - 3 activities, provably maximal by the exchange argument
(any optimal solution can be modified to include the earliest-ending activity without losing count)

1.6 Simple Analogy

Greedy is like **choosing your next meeting slot by always picking whichever available option ends soonest** — freeing up your calendar as early as possible for future opportunities — a strategy that’s provably optimal for “maximize number of meetings attended,” but would be a bad, unproven guess for a different scheduling goal (like “maximize total meeting value”) without re-deriving a new proof.

1.7 When Should I Immediately Think About Using This Pattern?

- “Maximum number of non-overlapping intervals,” “minimum number of resources.”
 - Problems where sorting by some key (end time, ratio, size) followed by a single pass gives an intuitively “obvious” answer — but only proceed with Greedy if you can articulate *why* it’s provably correct.
 - “Assign cookies,” “gas station,” “jump game” (reachability-style greedy).
 - Huffman coding, minimum spanning tree (Kruskal’s/Prim’s — both greedy with Union-Find/heap support).
-

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“maximum number of non-overlapping”	Interval greedy (sort by end time)
“minimum number of...” (resources, jumps, platforms)	Greedy with a specific exchange-argument structure
“assign,” “distribute” with a fairness/matching criterion	Two-pointer greedy after sorting
“at every step, choose the best available option”	Direct greedy framing
“fractional” (Fractional Knapsack)	Greedy works when items are divisible, unlike 0/1 Knapsack

2.2 Hidden Hints

A problem that “smells like” DP or Backtracking but has a suspiciously simple, single-pass-after-sorting solution that seems to always work on small examples — the interviewer is testing whether you can find (and prove) the greedy insight rather than defaulting to a more complex, over-engineered DP solution.

2.3 Interview Clues

Interviewer asks “can you prove this greedy choice is always safe?” — this is the single most direct signal that Greedy is expected AND that a rigorous justification (not just intuition) is required.

2.4 Common Trick Words

“Optimal,” “maximum/minimum,” combined with a natural sort key (end time, ratio, deadline) — but be wary: many of these same trick words also apply to DP problems, so the real differentiator is whether you can find and prove an exchange argument.

2.5 What Interviewers Expect

An explicit exchange-argument proof sketch (not just “this seems right”), correct sort-key selection, and — critically — recognition of when Greedy does **NOT** work (e.g., 0/1 Knapsack, where greedy by value/weight ratio fails because items aren’t divisible) versus when it does (Fractional Knapsack, where the same ratio-based greedy IS provably optimal).

2.6 When NOT To Use This Pattern

- You **cannot construct or verify** an exchange-argument proof — if the locally-best choice's safety isn't provable, Greedy is just an unverified heuristic that may produce wrong answers on some inputs (as in the {1,3,4} coin change counterexample).
 - The problem has **overlapping subproblems** requiring reconsideration of past choices based on future information — that's DP's territory, not Greedy's.
 - Items are **discrete/indivisible** in a way that breaks the exchange argument (0/1 Knapsack: greedy by ratio fails; must use DP instead).
-

SECTION 3 — Decision Framework

Does the problem have a natural "locally best choice" at each step (sort key, obvious criterion)?

Yes

Can you PROVE (via exchange argument) that this local choice never harms the global optimum?

Yes → USE GREEDY (typically $O(n \log n)$, sort-then-single-pass)

No - can't prove it, or a counterexample exists

Do OVERLAPPING SUBPROBLEMS exist, requiring reconsideration of past choices?

Yes → USE DYNAMIC PROGRAMMING instead (Patterns #22-26)

No

Do you need to explore ALL possibilities, not just the greedy path?

Yes → USE BACKTRACKING instead (Pattern #12)

Why: The single most important discipline in this pattern is refusing to apply Greedy without a proof — an unproven “greedy-looking” heuristic can be silently wrong (as the classic denomination counterexample demonstrates), and interviewers specifically probe for this proof to distinguish genuine algorithmic understanding from pattern-matched guessing.

SECTION 4 — Why This Pattern Works

The Exchange Argument (General Form): To prove a greedy choice is safe, assume for contradiction that some optimal solution OPT does NOT make the greedy choice G at the first point of divergence. Show that OPT can be modified — by “exchanging” its choice for G — into a new solution OPT' that is **at least as good** as OPT (same or better objective value) and still valid (satisfies all constraints). Since OPT was assumed optimal and OPT' is at least as good, OPT' is also optimal, and it agrees with the greedy choice at this step — by induction, this argument can be repeated at every subsequent step, showing that some optimal solution matches the greedy algorithm's choices entirely.

Correctness Proof (Activity Selection, as a concrete example): *Claim:* always selecting the available activity with the earliest end time is safe. *Proof:* let OPT be an optimal solution, and let a be the activity with the globally earliest end time. If a ∉ OPT, done. If a ∈ OPT, let b be the first activity in OPT (in start-time order). Since a has the earliest end time among ALL activities, end(a) ≤ end(b). Replacing b with a in OPT cannot cause any conflict with OPT's other activities (since a ends no later than b did, everything that didn't conflict with b still doesn't conflict with a), and

the resulting solution has the same size — so it's still optimal, and now includes a. This proves the greedy choice is always safe, and by induction (applying the same argument to the remaining subproblem after removing conflicting activities), the entire greedy algorithm is optimal. **QED.**

Why this differs from DP: DP handles problems where the optimal choice at a step genuinely depends on information not yet known (requiring consideration of multiple possibilities, cached for reuse) — Greedy only works when a rigorous proof shows this dependency doesn't actually exist, i.e., the locally best choice is ALWAYS part of some globally optimal solution, regardless of future information.

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework

1. Identify a **sort key/local criterion** that intuitively seems like “the best choice to make first” (earliest end time, best ratio, smallest deadline, etc.).
2. **Prove** (via exchange argument) that committing to this locally-best choice never harms the global optimum.
3. Sort the input by this key.
4. Perform a **single pass**, greedily selecting/rejecting/accumulating based on the criterion, never reconsidering past choices.
5. Return the accumulated result.

5.2 General Template — Interval Scheduling (Maximize Non-Overlapping Count)

```
function maxNonOverlapping(intervals):
    sort intervals by END time ascending
    count = 0
    lastEnd = -infinity

    for interval in intervals:
        if interval.start >= lastEnd:
            count = count + 1
            lastEnd = interval.end

    return count
```

5.3 General Template — Fractional Knapsack

```
function fractionalKnapsack(items, capacity):           # items: (weight, value)
    sort items by (value / weight) DESCENDING
    totalValue = 0
    remainingCapacity = capacity

    for item in items:
        if item.weight <= remainingCapacity:
            totalValue += item.value
            remainingCapacity -= item.weight
        else:
            fraction = remainingCapacity / item.weight
            totalValue += item.value * fraction
            remainingCapacity = 0
            break

    return totalValue
```

5.4 Interview Thinking Process

1. "I'll look for a natural sort key that makes an 'obviously best' local choice each step."
2. "Before committing to this approach, I'll construct an exchange-argument proof: can I show that any optimal solution can be modified to include this greedy choice without becoming worse?"
3. "If I can't find such a proof, or I find a counterexample, I'll pivot to DP instead."
4. "Once proven, I'll sort by the identified key and perform a single greedy pass."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n \log n)$ (dominated by the initial sort)	$O(1)$ to $O(n)$ (depending on whether sorting is in-place)	The greedy pass itself is $O(n)$; sorting dominates
Average Case	$O(n \log n)$	$O(1)$ to $O(n)$	Same regardless of data distribution
Best Case	$O(n \log n)$ (sort is generally unavoidable unless pre-sorted)	$O(1)$ to $O(n)$	Even trivial inputs need the sort (unless explicitly pre-sorted)
Amortized	$O(n \log n)$ total — single pass after sort, no repeated work	$O(1)$ to $O(n)$	This is the entire efficiency advantage over DP/Backtracking for problems where Greedy provably applies

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty input	No items/intervals	Return 0 or an empty result immediately
Single item	One interval/item	Trivially select it (no conflicts possible)
All items identical	Same interval repeated	Sort stability doesn't affect correctness for these problems (equal keys can be processed in any relative order)
Ties in the sort key	Multiple items with the same end time/ratio	Verify the exchange argument still holds with ties (usually yes, but confirm)
Items that can never fit/be selected	An interval far outside all others, still valid	Correctly included on its own if it doesn't conflict with anything

Edge Case	Example	Handling
Greedy criterion misidentified	Applying wrong sort key (e.g., sort by start time instead of end time for interval scheduling)	Silently produces suboptimal results — must re-derive the correct key via the exchange argument

Common mistakes: choosing the wrong sort key without verifying via the exchange argument (e.g., sorting activities by start time or duration instead of end time — both are tempting but WRONG for maximizing non-overlapping count); applying a greedy approach to a problem that actually requires DP (0/1 Knapsack), silently producing an incorrect (usually suboptimal) answer that looks plausible.

SECTION 8 — Pros & Cons

Advantages: Extremely efficient (typically $O(n \log n)$) when provably applicable; simple, clean single-pass implementation once the correct criterion is identified. **Disadvantages:** Only works when a rigorous exchange-argument proof exists — applying it without this proof is a dangerous, unverified guess; identifying the *correct* sort key/criterion is sometimes non-obvious and requires careful proof-construction, not just intuition. **Trade-offs:** Greedy ($O(n \log n)$, provably optimal when applicable) vs. DP ($O(n^2)$ or worse, always correct for overlapping-subproblem structures, no proof-of-local-optimality needed) — Greedy is strictly better when it applies, but silently wrong when misapplied. **Limitations:** Doesn't generalize — every new greedy problem requires its own fresh exchange-argument proof; no “one greedy technique fits all.” **Inefficient when:** N/A when correctly applicable — the concern with Greedy is never inefficiency, but incorrect applicability.

SECTION 9 — Real World Applications

Domain	Application
Operating Systems	CPU scheduling algorithms (Shortest Job First is a greedy algorithm, provably optimal for minimizing average waiting time under certain assumptions)
Networking	Huffman coding (greedy, always merges the two least-frequent nodes) is used in data compression (e.g., within DEFLATE/gzip)
Kruskal's/Prim's MST Algorithms	Both are greedy algorithms (with Union-Find or heap support) for minimum spanning tree construction, used in network design
Financial Systems	Greedy algorithms for certain simplified currency-exchange / arbitrage-avoidance heuristics (though exact solutions may need Bellman-Ford, Pattern #21)
Amazon/Uber Logistics	Greedy nearest-neighbor heuristics for real-time dispatch decisions when exact optimization (Bitmask DP/TSP) is infeasible at scale
Meeting Room Booking Systems	Greedy interval-scheduling algorithms for maximizing meeting room utilization

Domain	Application
Cache Replacement Policies	Certain cache eviction heuristics (e.g., a simplified “evict the one needed furthest in the future,” Belady’s algorithm — provably optimal, though requiring future knowledge) are greedy
Load Balancing	Greedy “assign to least-loaded server” heuristics for real-time request distribution
Data Compression	Huffman coding remains a canonical, widely-deployed greedy algorithm application
Currency/Vending Machines	Greedy coin-dispensing algorithms (correct for well-designed “canonical” currency systems, though not universally correct for arbitrary denominations)

SECTION 10 — Interview Strategy

How seniors answer: They explicitly state the candidate greedy criterion and immediately follow with an exchange-argument proof sketch before writing any code, and they proactively distinguish the problem from superficially similar DP problems where greedy would NOT work (explaining why the exchange argument fails there).

How juniors answer: They often propose a greedy approach that “seems right” based on a couple of test cases, without constructing or checking a rigorous proof — this works on easy problems but fails on adversarial test cases specifically designed to break unproven greedy heuristics.

Typical follow-ups: “Can you prove this greedy choice is always safe?” “Can you construct a counterexample where greedy fails, to make sure you understand why this problem is different?” “What’s the difference between this and the 0/1 Knapsack problem, where a similar-looking greedy approach fails?”

Optimization questions: “Can you avoid the $O(n \log n)$ sort if the input is already sorted?” (Yes — then the greedy pass alone is $O(n)$.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Interval Scheduling (Maximize Count)	Sort by end time, greedily select non-overlapping	Non-overlapping Intervals, Minimum Number of Arrows to Burst Balloons
Fractional Knapsack	Sort by value/weight ratio, take greedily (divisible items)	Fractional Knapsack (classic)
Huffman Coding	Repeatedly merge two least-frequent nodes (heap-based greedy)	Huffman Encoding (classic)
Minimum Spanning Tree	Greedy edge selection with cycle avoidance (Union-Find)	Kruskal’s Algorithm (Pattern #19 cross-reference)
Jump Game / Reachability Greedy	Track the furthest reachable index greedily	Jump Game, Jump Game II

Variation	Description	Example
Two-Pointer Greedy (Assignment/Matching)	Sort both sides, greedily match	Assign Cookies, Boats to Save People
Deadline-Based Greedy	Sort by deadline, greedily schedule with a priority structure	Job Sequencing with Deadlines, Task Scheduler

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Dynamic Programming	Considers overlapping subproblems, doesn't require an upfront local-optimality proof, always correct when applicable	No exchange-argument proof exists, or a counterexample to greedy is found
Backtracking	Explores all possibilities, no proof of local optimality needed (or possible)	Need all solutions, or no greedy/DP structure exists
Divide and Conquer	Splits into independent subproblems, combines results, no "local choice" framing	Subproblems are naturally independent, not sequentially chosen

Comparison Table

Aspect	Greedy	Dynamic Programming	Backtracking
Requires proof of local optimality	Yes (exchange argument)	No	No
Reconsiders past choices	Never	Implicitly (via subproblem table)	Yes (explicit undo)
Time complexity	$O(n \log n)$ typically	Polynomial (varies)	Exponential
Risk if misapplied	Silently wrong answer	N/A (correct by construction)	N/A (exhaustive, correct)

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct, well-known greedy criterion	Assign Cookies, Lemonade Change
Medium	Requires deriving a non-obvious sort key/proof	Non-overlapping Intervals, Jump Game, Gas Station

Difficulty	Characteristics	Examples
Hard	Multi-criteria greedy, combined with heap/Union-Find Advanced exchange arguments, greedy + advanced structure combinations	Task Scheduler, Minimum Number of Arrows to Burst
Very Hard		Balloons, Candy Course Schedule III, Minimum Cost to Hire K Workers, IPO

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Assign Cookies	Easy	Amazon, Meta	Two-pointer greedy matching	Foundational greedy matching
2	Lemonade Change	Easy	Amazon	Direct simulation greedy	Basic greedy simulation
3	Non-overlapping Intervals	Medium	Amazon, Meta, Google	Interval scheduling (sort by end time)	Foundational exchange argument
4	Minimum Number of Arrows to Burst Balloons	Medium	Amazon, Google	Interval greedy (sort by end)	Interval greedy reinforcement
5	Jump Game	Medium	Amazon, Meta, Google	Reachability greedy (track furthest reach)	Reachability greedy mastery
6	Jump Game II	Medium	Amazon, Meta, Google	Minimum jumps via greedy level expansion	Advanced reachability greedy
7	Gas Station	Medium	Amazon, Meta, Google	Greedy with total-sum feasibility argument	Advanced greedy proof (total sum argument)
8	Candy	Hard	Amazon, Google, Microsoft	Two-pass greedy (left-to-right, right-to-left)	Advanced dual-pass greedy
9	Task Scheduler	Medium	Amazon, Meta, Google	Greedy + heap for frequency-based scheduling	Greedy + heap combination

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
10	Partition Labels	Medium	Amazon, Meta	Greedy with last-occurrence tracking	Boundary-tracking greedy
11	Fractional Knapsack (classic)	Medium	Amazon, Google (conceptually asked)	Ratio-based greedy (contrast with 0/1 Knapsack)	Divisibility-dependent greedy
12	Huffman Encoding (classic)	Medium/Hard	Google, Amazon (conceptually asked)	Heap-based greedy merging	Heap + greedy combination
13	Minimum Cost to Connect Sticks	Medium	Amazon, Google	Heap-based greedy merging (Huffman-style)	Heap + greedy application
14	Boats to Save People	Medium	Amazon, Google	Two-pointer greedy matching	Two-pointer greedy variant
15	Queue Re-construction by Height	Medium	Amazon, Google	Sort + greedy insertion	Advanced sort-key derivation
16	Course Schedule III	Hard	Google, Amazon	Greedy + heap for deadline-based scheduling	Advanced greedy + heap
17	Job Sequencing with Deadlines (classic)	Medium	Amazon, Google (conceptually asked)	Greedy + Union-Find/priority structure	Deadline-based greedy
18	Minimum Number of Platforms (classic)	Medium	Amazon, Google (conceptually asked)	Sort + two-pointer greedy sweep	Sweep-based greedy
19	IPO	Hard	Amazon, Google	Greedy + heap for capital-constrained optimization	Advanced greedy + heap
20	Minimum Cost to Hire K Workers	Hard	Google, Amazon	Sort + max-heap greedy with ratio constraint	Advanced ratio-based greedy
21	Two City Scheduling	Medium	Amazon, Google	Sort by cost-difference greedy	Difference-based greedy

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
22	Maximum Units on a Truck	Easy	Amazon	Ratio-based greedy (simplified fractional knapsack)	Basic ratio greedy
23	Reorganize String	Medium	Amazon, Google	Greedy + heap for frequency-based arrangement	Greedy + heap combination
24	Remove K Digits (contrast, monotonic stack)	Medium	Amazon, Google	Contrast: Monotonic Stack, related greedy-adjacent technique	Cross-pattern awareness
25	Car Pooling (contrast, difference array)	Medium	Amazon, Google	Contrast: Prefix Sum/Difference Array, not pure greedy	Pattern-boundary awareness
26	Maximum Performance of a Team	Hard	Google, Amazon	Sort + heap greedy with ratio constraint	Advanced ratio-based greedy
27	Split Array Largest Sum (contrast, Binary Search)	Hard	Google, Amazon	Contrast: Binary Search on Answer, not pure greedy	Pattern-boundary awareness
28	Video Stitching	Medium	Google	Greedy interval covering	Interval-covering greedy
29	Maximum Length of Pair Chain	Medium	Amazon, Google	Greedy interval chaining (sort by end)	Interval greedy reinforcement
30	Advantage Shuffle	Medium	Google, Amazon	Two-pointer greedy matching (advantage-based)	Advanced matching greedy

SECTION 15 – Common Mistakes

1. Choosing a plausible-but-wrong sort key without verifying it against an exchange-argument proof (e.g., sorting activities by start time or duration instead of end time). *Fix:* always attempt to construct the proof before committing to a specific criterion.
2. Applying greedy to a problem that actually requires DP (0/1 Knapsack being the canonical counterexample to a ratio-based greedy), producing a silently suboptimal answer. *Fix:*

always check whether items/choices are divisible/reusable in a way that preserves the exchange argument, or whether they're discrete in a way that breaks it.

3. Assuming a greedy approach that works on a few test cases generalizes to all inputs, without constructing a rigorous proof or actively searching for counterexamples. *Fix:* explicitly attempt to break your own greedy hypothesis with adversarial examples before finalizing.
4. Forgetting to sort before the greedy pass (or sorting by the wrong direction — ascending vs descending). *Fix:* always explicitly state and verify the sort direction matches the proof's requirements.
5. Not recognizing when a problem needs TWO greedy passes (e.g., Candy — left-to-right then right-to-left) rather than a single pass. *Fix:* consider whether a single directional pass can capture all the problem's constraints, or if multiple passes/directions are needed.

Why people fail: Greedy's biggest risk is that incorrect applications often still “look plausible” and pass several test cases, because the greedy choice frequently coincides with the optimal one on simple/random inputs — the failures only surface on carefully constructed adversarial inputs, which is exactly what distinguishes rigorous proof-based reasoning from pattern-matched intuition, and exactly what interviewers test for with edge-case follow-ups.

SECTION 16 — Optimization Techniques

- **Time:** If the input is already sorted (or can be assumed so), skip the $O(n \log n)$ sort and the greedy pass alone is $O(n)$.
 - **Space:** Most greedy algorithms need only $O(1)$ additional space beyond the sort itself (in-place sorting) — a natural efficiency advantage over DP's typically larger space requirements.
 - **Readability:** Explicitly comment the exchange-argument justification directly above the sort/greedy logic in code — this documents *why* the approach is correct, not just *what* it does.
 - **Interview performance:** Always state the sort key and its proof BEFORE writing code — this single habit is the clearest signal of rigorous (not guessed) greedy reasoning, and it's the exact thing interviewers are listening for.
-

SECTION 17 — Multiple Language Templates

Java

```
public int eraseOverlapIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0;
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);
    int count = 1;
    int lastEnd = intervals[0][1];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= lastEnd) {
            count++;
            lastEnd = intervals[i][1];
        }
    }
    return intervals.length - count;
}
```

JavaScript

```
function eraseOverlapIntervals(intervals) {
    if (intervals.length === 0) return 0;
    intervals.sort((a, b) => a[1] - b[1]);
    let count = 1, lastEnd = intervals[0][1];
    for (let i = 1; i < intervals.length; i++) {
        if (intervals[i][0] >= lastEnd) {
            count++;
            lastEnd = intervals[i][1];
        }
    }
    return intervals.length - count;
}
```

PHP

```
function eraseOverlapIntervals(array $intervals): int {
    if (empty($intervals)) return 0;
    usort($intervals, fn($a, $b) => $a[1] <=> $b[1]);
    $count = 1; $lastEnd = $intervals[0][1];
    for ($i = 1; $i < count($intervals); $i++) {
        if ($intervals[$i][0] >= $lastEnd) {
            $count++;
            $lastEnd = $intervals[$i][1];
        }
    }
    return count($intervals) - $count;
}
```

Python

```
def erase_overlap_intervals(intervals):
    if not intervals:
        return 0
    intervals.sort(key=lambda x: x[1])
    count = 1
    last_end = intervals[0][1]
    for start, end in intervals[1:]:
        if start >= last_end:
            count += 1
            last_end = end
    return len(intervals) - count
```

Go

```
func eraseOverlapIntervals(intervals [][]int) int {
    if len(intervals) == 0 {
        return 0
    }
    sort.Slice(intervals, func(i, j int) bool { return intervals[i][1] < intervals[j][1] })
    count := 1
    lastEnd := intervals[0][1]
```

```

    for i := 1; i < len(intervals); i++ {
        if intervals[i][0] >= lastEnd {
            count++
            lastEnd = intervals[i][1]
        }
    }
    return len(intervals) - count
}

```

C++

```

int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    if (intervals.empty()) return 0;
    sort(intervals.begin(), intervals.end(), [](auto& a, auto& b) { return a[1] < b[1]; });
    int count = 1, lastEnd = intervals[0][1];
    for (int i = 1; i < (int)intervals.size(); i++) {
        if (intervals[i][0] >= lastEnd) {
            count++;
            lastEnd = intervals[i][1];
        }
    }
    return (int)intervals.size() - count;
}

```

SECTION 18 — Dry Runs

Small Input

intervals = [[1,2],[2,3],[3,4],[1,3]] (Non-overlapping Intervals — minimum removals)

Sort by end: [[1,2],[2,3],[1,3],[3,4]]

count=1, lastEnd=2 (from [1,2])

[2,3]: start=2 >= lastEnd=2 → count=2, lastEnd=3

[1,3]: start=1 < lastEnd=3 → skip (overlap, would need removal)

[3,4]: start=3 >= lastEnd=3 → count=3, lastEnd=4

Total intervals=4, count kept=3 → removals needed = 4-3 = 1 (remove [1,3])

Large Input (Conceptual)

For 10^5 intervals, the greedy approach costs $O(n \log n) \approx 1.7 \times 10^6$ operations (sort-dominated) — versus an exponential/DP-based brute-force alternative that would be vastly more expensive, illustrating Greedy's efficiency advantage when provably applicable.

Corner Case

intervals = [[1,2]] (single interval): sort trivial, count=1, loop doesn't execute (only one interval) → removals needed = 1-1=0, correctly requiring no removals for a single, non-conflicting interval.

SECTION 19 — Advanced Concepts

- **Matroid Theory (why greedy works, generalized):** many classic greedy problems (MST via Kruskal's, certain scheduling problems) can be formally shown to be optimizing over a **matroid** — an abstract structure generalizing “linear independence” — and a theorem states that greedy algorithms are always optimal for matroid-structured optimization problems. This is a Staff-level theoretical framework explaining *why* greedy works for an entire class of problems at once, rather than needing a fresh proof for every individual problem.
 - **Greedy + Heap combinations (Huffman Coding, Task Scheduler, IPO):** many “hard” greedy problems require a heap to efficiently maintain “the current best available choice” as the greedy process unfolds and the set of available choices changes dynamically — recognizing this combination (Greedy + Heap, Pattern #17) is a common advanced technique.
 - **Total-sum feasibility arguments (Gas Station):** some greedy proofs don't rely on a simple pairwise exchange argument, but instead on a global feasibility property (e.g., “if the total gas is enough to complete the circuit, there must exist a valid starting point, and it can be found greedily by tracking the running deficit”) — a different but equally rigorous style of greedy proof.
 - **Counterexample construction as a validation habit:** before finalizing any greedy solution, actively try to construct a small adversarial input that might break it — this proactive counterexample-hunting is what separates confident, verified greedy solutions from unproven guesses.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize that Greedy's entire value and entire risk stem from the same source: **it commits early, with no way to reconsider, and is therefore only as trustworthy as its accompanying proof.** They actively distinguish between reaching for Greedy because it's provably correct (rigorous) versus reaching for it because it “feels right” (dangerous), and they know the canonical DP-vs-Greedy contrast (0/1 Knapsack vs Fractional Knapsack) cold, using it as a mental litmus test for any new problem: “is this discrete/indivisible (likely needs DP) or continuous/divisible/exchange-argument-provable (Greedy may apply)?” Interviewers evaluate whether a candidate proactively constructs or requests validation of an exchange argument, rather than presenting an unproven greedy heuristic as if it were self-evidently correct.

SECTION 21 — Cheat Sheet

PATTERN: Greedy Algorithms

RECOGNIZE: "maximum/minimum," natural sort key, "at every step choose the best available option"

TEMPLATE:

```
sort input by identified key (PROVEN via exchange argument)
for each element in sorted order:
    if it fits the current state / doesn't conflict: take it, update state
    else: skip it
return accumulated result
```

COMPLEXITY: $O(n \log n)$ typically (sort-dominated)

KEY PROOF: exchange argument - show any optimal solution can be modified to include the greedy choice with

WATCH FOR: unproven greedy heuristics (silently wrong), wrong sort key/direction, problems needing multiple

DOESN'T APPLY WHEN: no exchange-argument proof exists (use DP), items are discrete/indivisible in a way that

SECTION 22 — Revision Notes (5-Minute Review)

- Greedy makes the locally best choice at every step, never reconsidering — only correct WITH a proof (exchange argument).
- Exchange argument: show any optimal solution can be modified to match the greedy choice without becoming worse.
- Fractional Knapsack (divisible items) → greedy works; 0/1 Knapsack (indivisible) → greedy fails, needs DP.
- Interval scheduling: sort by END time (not start time or duration) — this specific key is what the exchange argument requires.
- Always actively try to construct a counterexample before trusting a greedy hypothesis.
- Greedy + Heap combinations handle problems where “current best choice” changes dynamically (Huffman, Task Scheduler).

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic greedy simulation	Assign Cookies (455), Lemonade Change (860)
Intermediate	Interval scheduling, reachability	Non-overlapping Intervals (435), Jump Game (55), Gas Station (134)
Advanced	Greedy + heap, dual-pass greedy	Task Scheduler (621), Candy (135), Partition Labels (763)
Expert	Advanced ratio-based and constrained greedy	IPO (502), Minimum Cost to Hire K Workers (857), Course Schedule III (630)

SECTION 24 — Memory Tricks

- **Mnemonic:** “**Prove Before Proceed**” (PBP) — always Prove the exchange argument Before Proceeding with a greedy implementation.
- **Visualization:** **Always picking the meeting that ends soonest** to free up your calendar for the most future opportunities — a provably optimal strategy for maximizing meetings attended.
- **Recognition shortcut:** “Maximum/minimum” + an obvious sort key → Greedy candidate; but ALWAYS verify with an exchange argument before trusting it.

SECTION 25 — Final Summary

Greedy algorithms make irrevocable, locally-optimal choices at every step, achieving efficient $O(n \log n)$ solutions — but only when a rigorous exchange-argument proof demonstrates that this local optimality never sacrifices global optimality. The single most important thing to remember forever: **never trust a greedy approach just because it “seems right” or passes a few test cases — always construct (or at least seriously attempt) the exchange-argument proof, and actively hunt for counterexamples before finalizing, since the canonical contrast between Fractional Knapsack (greedy works, items divisible) and 0/1 Knapsack (greedy fails, items discrete, needs DP) is exactly the kind of subtle boundary where confident-but-wrong greedy heuristics most commonly and dangerously fail.**